Computational Complexity Comparison of FL Baselines

1 Notation

Table 1: Complexity-analysis notation

Symbol	Meaning
d	Total number of model parameters
N	Total number of clients
K	Number of clients selected per round ($K \leq N$)
T	Total number of global rounds
E	Number of local training epochs per round
$\lvert\mathcal{D}_i\rvert$	Size of client i's local dataset
n	CKKS/RLWE polynomial modulus degree; slot count is $n/2$
J	Number of packs/ciphertexts for the full model, $J = \lceil 2d/n \rceil$ for merged CKKS packing
k	Number of consensus packs selected by voting ($k \leq J$)
s	Per-client sparsity ratio in FedPHE, $0 < s \leq 1$
R	Prefetch window in ECPFL (rounds amortized into one decryption)
L	Number of RNS moduli (levels) in multi-key CKKS ciphertexts
$\mathcal{O}_{\text{ntt}}$	Cost of one NTT/INTT, $\mathcal{O}_{\text{ntt}} = \mathcal{O}(n \log n)$
$\mathcal{O}_{\text{he-add}}$	Cost of one ciphertext addition, $\mathcal{O}_{\text{he-add}} = \mathcal{O}(n)$
$\mathcal{O}_{\text{he-smult}}$	Cost of one ciphertext scalar multiplication, $\mathcal{O}_{\text{he-smult}} = \mathcal{O}(n)$

All complexities are stated **per global round** unless otherwise noted. The local-training cost $\mathcal{C}_{\text{train}} = O(E\,|\mathcal{D}_i|\,d)$ is identical for every scheme and is therefore omitted from the dominant cryptographic comparisons, but it is included for completeness in section 3.

2 Complexity Summary

The bottom-line comparison is:

- **FedAvg+CKKS** and **ESAFL** have the lowest server aggregation cost, $\mathcal{O}(Kd)$, because ciphertexts do not grow during aggregation.
- **MASER** pays a multiplicative K penalty on the server ($\mathcal{O}(K^2 dL)$) because its multi-key ciphertexts grow from 2 to $K+1$ components when homomorphically summed.
- **FedPHE** keeps server aggregation at $\mathcal{O}(Ksd \log n)$, but the $\log n$ factor appears because it uses CKKS ciphertext-plaintext multiplications for weighted aggregation.
- **ECPFL** adds voting/perturbation overhead $\mathcal{O}(NJ)$ per active client and $\mathcal{O}(NKJ)$ at the KGC, but reduces the encryption/upload volume to $k \leq J$ packs via consensus sparsity and amortizes decryption over R rounds via prefetching.

3 Detailed Derivations

For each scheme we derive the complexity from its core design logic, not from simulation-code shortcuts.

3.1 FedAvg+CKKS

Design. Standard FedAvg with a single CKKS key pair. All selected clients encrypt their local updates; the server aggregates ciphertexts homomorphically and decrypts the global model for evaluation.

Let $J = \lceil 2d/n \rceil$ be the number of ciphertexts needed for the full model under merged slot packing.

Client side. Each selected client

1. downloads J ciphertexts and decrypts them: J CKKS decryptions;
2. trains locally and computes the model delta;
3. encrypts the delta into J ciphertexts: J CKKS encryptions.

Each CKKS encryption or decryption costs $\mathcal{O}_{\text{ntt}} = \mathcal{O}(n \log n)$. Therefore

$$\boxed{T_{\text{client}}^{\text{FedAvg+CKKS}} = 2J \cdot \mathcal{O}(n \log n) + \mathcal{O}(d) = \mathcal{O}(d \log n).} \tag{1}$$

Server side. The server receives $K \cdot J$ ciphertexts. For each of the J packs it computes a weighted average over K ciphertexts, which needs K scalar multiplications and $K - 1$ additions. Scalar mult and addition are both $\mathcal{O}(n)$. The server also decrypts J ciphertexts for evaluation. Hence

$$\boxed{T_{\text{server}}^{\text{FedAvg+CKKS}} = J \cdot K \cdot \mathcal{O}(n) + J \cdot \mathcal{O}(n \log n) = \mathcal{O}(Kd) + \mathcal{O}(d \log n).} \tag{2}$$

KGC. None; the single secret key is generated once and shared with the server.

3.2 MASER

Design. MASER uses multi-key CKKS (xMK-CKKS). Each client keeps its own secret key sk_i; a Key Manager aggregates the public-key components into a joint public key pk. Clients encrypt only parameters selected by a global majority-vote mask. The server sums ciphertexts and clients collectively perform partial decryption.

Let m be the number of parameters selected by the global mask and $J = \lceil m/(n/2) \rceil$ the number of slices/ciphertexts. Let d_w be the number of weight parameters used in mask voting.

Client side (per effective global round). MASER uses three sub-rounds (mask, encrypt, decrypt) per global update. Each client performs:

1. local training and magnitude-based local mask generation: $\mathcal{O}(d_w)$;
2. encrypt J slices under the aggregated public key: J CKKS encryptions at $\mathcal{O}(n \log n + nL)$ each;
3. partial-decrypt J aggregated multi-key ciphertexts: J partial decryptions at $\mathcal{O}(nL)$ each.

Thus

$$\boxed{T_{\text{client}}^{\text{MASER}} = J \cdot \mathcal{O}(n \log n + nL) + \mathcal{O}(d_w) = \mathcal{O}(d \log n + d_w).} \tag{3}$$

Server side (per effective global round). The server

1. aggregates K local masks: $\mathcal{O}(Kd_w)$;
2. homomorphically sums K ciphertexts for each of J slices. Because xMK-CKKS ciphertexts grow from 2 components to $K+1$ components, the cost of summing K ciphertexts is $\mathcal{O}(K^2nL)$ per slice;
3. merges K partial-decryption shares and decodes each of J slices: $\mathcal{O}(KnL + n\log n)$ per slice.

Therefore

$$\boxed{T_{\text{server}}^{\text{MASER}} = J \cdot \mathcal{O}(K^2nL) + J \cdot \mathcal{O}(KnL + n\log n) + \mathcal{O}(Kd_w) = \mathcal{O}(K^2dL + KdL + d\log n + Kd_w).} \tag{4}$$

The dominant term is $\mathcal{O}(K^2dL)$.

KGC / Key Manager. One-time aggregation of N public keys: $\mathcal{O}(NnL)$.

3.3 FedPHE

Design. FedPHE uses packed single-key CKKS (optionally Paillier/BFV). Each client sparsifies its update to an s fraction of packs, encrypts the selected packs, and uploads a binary mask. The server performs ciphertext-plaintext multiplication for contribution-aware weighted aggregation. Decryption is done at the clients.

Let $J = \lceil 2d/n \rceil$ and let each client upload sJ packs in expectation.

Client side. Each selected client

1. trains locally;
2. scores and selects top-sJ packs: $\mathcal{O}(d + J\log J)$;
3. encrypts sJ packs: $sJ \cdot \mathcal{O}(n\log n)$;
4. downloads and decrypts the global model in J packs: $J \cdot \mathcal{O}(n\log n)$;
5. fills zero positions from local parameters: $\mathcal{O}(d)$.

Hence

$$\boxed{T_{\text{client}}^{\text{FedPHE}} = sJ \cdot \mathcal{O}(n\log n) + J \cdot \mathcal{O}(n\log n) + \mathcal{O}(d) = \mathcal{O}(sd\log n + d\log n) = \mathcal{O}(d\log n).} \tag{5}$$

Server side. The server

1. aggregates K masks of length J: $\mathcal{O}(KJ)$;
2. for each active pack, performs a CKKS ciphertext-plaintext multiplication and addition. There are sJ active packs per client and K clients, so KsJ mult/add operations at $\mathcal{O}(n\log n)$ each.

Thus

$$\boxed{T_{\text{server}}^{\text{FedPHE}} = KsJ \cdot \mathcal{O}(n\log n) + \mathcal{O}(KJ) = \mathcal{O}(Ksd\log n).} \tag{6}$$

Optional client selection. MinHash-based selection costs $\mathcal{O}(Ld)$ per client and $\mathcal{O}(N^2L)$ server clustering, but it is typically run every several rounds, not every round.

KGC. None in the reference implementation; the secret key is shared with all clients.

3.4 ESAFL

Design. ESAFL uses a multi-client RLWE additively homomorphic scheme (ESHE). Each client has its own secret key s_i; the global decryption key is $S = \sum_i s_i$. A per-round public polynomial $a^{(t)}$ is generated locally by all parties from a common seed via PRPG. Ciphertexts are aggregated by simple addition, and decryption is one-step using S.

Let $J = \lceil 2d/n \rceil$.

Client side. Each selected client

1. trains locally;
2. computes PRPG output $a^{(t)}$: $\mathcal{O}(n)$;
3. encodes and encrypts the model update into J ciphertexts: $J \cdot \mathcal{O}(n \log n)$;
4. downloads and decrypts the aggregated ciphertext with S: $J \cdot \mathcal{O}(n \log n)$.

Therefore

$$\boxed{T_{\text{client}}^{\text{ESAFL}} = 2J \cdot \mathcal{O}(n \log n) + \mathcal{O}(n) + \mathcal{O}(d) = \mathcal{O}(d \log n).} \tag{7}$$

Server side. The server receives $K \cdot J$ ciphertexts and homomorphically sums them. Each addition is $\mathcal{O}(n)$. Hence

$$\boxed{T_{\text{server}}^{\text{ESAFL}} = KJ \cdot \mathcal{O}(n) = \mathcal{O}(Kd).} \tag{8}$$

KGC. One-time setup: generate N keys, compute $S = \sum_i s_i$, distribute seeds. No per-round KGC work.

3.5 ECPFL (Proposed)

Design. ECPFL builds on ESAFL and adds (i) pack-level voting with a global consensus set $\mathcal{K}^{(t)}$, (ii) pairwise perturbation for privacy-preserving vote aggregation, and (iii) ciphertext prefetching so inactive clients amortize decryption over R rounds.

Let $k = |\mathcal{K}^{(t)}|$ be the number of consensus packs, with $k \leq J$.

3.5.1 Active client

An active client performs:

1. local training: $\mathcal{O}(E|\mathcal{D}_i|d)$;
2. residual compensation $\tilde{\mathbf{g}}_i = \mathbf{g}_i + \mathbf{r}_i$: $\mathcal{O}(d)$;
3. score all J packs: $\mathcal{O}(d)$;
4. select local Top-k packs: $\mathcal{O}(J)$ with linear selection or $\mathcal{O}(J \log J)$ with sorting;
5. build raw vote bitmap $\mathbf{v}_i \in \{0,1\}^J$: $\mathcal{O}(J)$;
6. generate pairwise perturbation $\mathbf{z}_i^{(t)}$: $N-1$ PRG calls each of length J, costing $\mathcal{O}(NJ)$;
7. obfuscate vote $\hat{\mathbf{v}}_i = [\mathbf{v}_i + \mathbf{z}_i]_M$: $\mathcal{O}(J)$;
8. upload obfuscated vote: $\mathcal{O}(J)$ communication;
9. receive consensus set $\mathcal{K}^{(t)}$: $\mathcal{O}(J)$ or $\mathcal{O}(k)$ communication;
10. generate PRPG $a^{(t)}$: $\mathcal{O}(n)$;

11. encrypt k consensus packs: $k \cdot \mathcal{O}(n \log n)$;

12. update residual $\mathbf{r}_i^{(t)}$: $\mathcal{O}(d)$.

Adding the non-training terms gives

$$\boxed{T^{\text{ECPFL}}_{\text{client, active}} = \mathcal{O}(d + NJ) + k \cdot \mathcal{O}(n \log n) + \mathcal{O}(E|\mathcal{D}_i|d).} \tag{9}$$

3.5.2 Inactive / prefetch client

An inactive client participating in prefetching:

1. receives and stores/merges prefetch deltas for R rounds: $\mathcal{O}(Rk)$ ciphertext references and $\mathcal{O}(Rk)$ homomorphic additions if accumulation is implemented;

2. when selected, performs one CKKS decryption of the accumulated R-round composite ciphertext: $k \cdot \mathcal{O}(n \log n)$, amortized over R rounds.

Thus the amortized per-round prefetch cost is

$$\boxed{T^{\text{ECPFL}}_{\text{client, prefetch}} = \mathcal{O}(k) + \frac{k \cdot \mathcal{O}(n \log n)}{R} = \mathcal{O}\left(k + \frac{kn \log n}{R}\right).} \tag{10}$$

3.5.3 Server

The server performs:

1. client selection / scheduling (e.g., sticky or sketch-based): $\mathcal{O}(N \log N)$ or $\mathcal{O}(N)$, typically not on the critical training path;

2. receive K obfuscated votes of length J: $\mathcal{O}(KJ)$ communication;

3. ESP-Consensus: sum obfuscated votes and add KGC compensatory perturbation, $\mathcal{O}(KJ)$ modular additions; then compute Top-k consensus packs, $\mathcal{O}(J)$ or $\mathcal{O}(J \log J)$;

4. broadcast consensus set: $\mathcal{O}(J)$ or $\mathcal{O}(k)$ communication;

5. receive $K \cdot k$ ciphertexts;

6. homomorphically aggregate per consensus pack over K clients: Kk additions/scalar mults at $\mathcal{O}(n)$ each;

7. update encrypted global state: $\mathcal{O}(k)$;

8. compute prefetch delta. In the ESAFL path the aggregated ciphertext is reused as the delta: $\mathcal{O}(k)$. In the standard path the server decrypts the global state, computes a plaintext difference, and re-encrypts: $\mathcal{O}(Jn \log n)$;

9. queue update and per-client fetch preparation: $\mathcal{O}(k)$ per client needing a merge.

Ignoring the expensive standard-path fallback, the dominant server cost is

$$\boxed{T^{\text{ECPFL}}_{\text{server}} = \mathcal{O}(KJ) + \mathcal{O}(J \log J) + \mathcal{O}(Kk) + \mathcal{O}(k) = \mathcal{O}(KJ + J \log J + Kk).} \tag{11}$$

3.5.4 KGC

The KGC performs one-time setup and per-round compensation:

1. one-time: generate N keys, $S = \sum_i s_i$, PRPG seed B, and $\binom{N}{2}$ pairwise seeds: $\mathcal{O}(N^2)$;
2. per round: receive inactive set $\bar{\mathcal{S}}^{(t)}$;
3. compute compensatory perturbation $\mathbf{z}_{\text{comp}}^{(t)} = \sum_{k \in \bar{\mathcal{S}}^{(t)}} \mathbf{z}_k^{(t)}$: $\mathcal{O}((N-K)NJ)$, or equivalently $\mathcal{O}(NKJ)$ using the active-client optimization $\mathbf{z}_{\text{comp}}^{(t)} = -\sum_{i \in \mathcal{S}^{(t)}} \mathbf{z}_i^{(t)}$;
4. generate compensatory ciphertexts for each inactive client and each consensus pack: $(N-K)k$ CKKS encryptions at $\mathcal{O}(n \log n)$ each;
5. prefetch key-alignment masks: $\mathcal{O}(n \log n)$ per client per round.

Hence

$$\boxed{T_{\text{KGC}}^{\text{ECPFL}} = \mathcal{O}(NKJ) + (N-K)k \cdot \mathcal{O}(n \log n) + N \cdot \mathcal{O}(n \log n).} \tag{12}$$

4 Unified Comparison

The summary in table 2 expresses each scheme in its own notation, which makes direct comparison difficult. table 3 therefore uses a single set of metrics—dominant client cost, dominant server cost, KGC online cost, upload/download ciphertext counts, rounds per global update, dropout tolerance, and suitable deployment regime—to expose the trade-offs more clearly.

4.1 What the unified view reveals

- **Server aggregation.** ESAFL and FedAvg+CKKS have the cheapest server aggregation, $\mathcal{O}(Kd)$. ECPFL is similar when $k \ll J$, while FedPHE pays an extra $\log n$ factor because it uses CKKS multiplication for weighting. MASER is the most expensive server-side scheme due to growing multi-key ciphertexts ($\mathcal{O}(K^2 dL)$).
- **Client upload volume.** ECPFL and FedPHE reduce upload relative to full-model schemes: ECPFL uploads k consensus packs and FedPHE uploads sJ packs. For typical $k \approx sJ$, both achieve comparable upload savings, but ECPFL also coordinates clients to encrypt the same k packs, which simplifies server aggregation.
- **Client-side extra work.** ECPFL adds $\mathcal{O}(d + NJ)$ non-HE perturbation work per active client. This term dominates the client cost when $NJ \gg d \log n$, i.e., when the number of clients N is large relative to the model dimension d.
- **KGC online work.** Only ECPFL requires a trusted third party to perform online work ($\mathcal{O}(NKJ)$ compensatory perturbation and compensatory ciphertext generation). MASER's KGC is one-time; FedAvg+CKKS, FedPHE, and ESAFL have no online KGC.

4.2 Implications for cross-device FL

Cross-device FL is characterized by very large N (thousands to millions), small selected fractions K/N, limited client compute/battery, intermittent connectivity, and cheap cloud servers but no per-round trusted third party. Viewed through this lens, ECPFL has both advantages and limitations:

- **Advantage: reduced upload/download volume.** Consensus sparsity reduces the per-client upload from J packs to k packs, and prefetching amortizes the client decryption over R rounds. Both are attractive for bandwidth- and battery-constrained devices.

- **Advantage: partial dropout tolerance.** Compensatory ciphertexts allow the server to aggregate over any subset of active clients without requiring all clients to be online, unlike MASER or ESAFL.
- **Limitation: pairwise perturbation does not scale to large N.** The $\mathcal{O}(NJ)$ client-side perturbation generation and the $\mathcal{O}(NKJ)$ KGC compensatory computation grow with N. For N in the millions, this is prohibitive. The $\mathcal{O}(N^2)$ pairwise seed setup is also infeasible at cross-device scale.
- **Limitation: online KGC.** Cross-device FL typically avoids online trusted authorities. ECPFL's reliance on the KGC to compute and transmit $\mathbf{z}_{\text{comp}}^{(t)}$ and compensatory ciphertexts every round is a deployment obstacle.
- **Conclusion.** In its current form ECPFL is better suited to the **cross-silo** regime (tens to hundreds of stable clients, moderate N, trusted KGC) than to pure **cross-device** FL. To make it cross-device friendly, the pairwise vote-perturbation mechanism would need to be replaced by a more scalable approach, such as small-group pairwise masking, a lightweight secure aggregator (SecAgg), or accepting that the server learns aggregate vote tallies without cryptographic masking.

5 Discussion

- **Server aggregation cost.** FedAvg+CKKS, ESAFL, and ECPFL enjoy ciphertext-size-preserving aggregation, so server HE work is linear in K (or Kk/J for ECPFL). MASER's multi-key ciphertexts grow with the number of contributors, producing a $\mathcal{O}(K^2)$ factor.
- **Client encryption cost.** ECPFL encrypts only k consensus packs per client instead of J, giving k/J savings on the dominant HE encryption term relative to ESAFL/FedAvg+CKKS. FedPHE achieves similar savings with per-client TopK sparsity but pays $\log n$ per aggregation operation because it uses CKKS multiplication for weighting.
- **Prefetching trade-off.** Prefetching adds $\mathcal{O}(k)$ server-side HE work per round. For clients, the decryption cost is $\mathcal{O}(kn \log n)$ but is paid only once every R rounds, giving an amortized $\mathcal{O}(kn \log n/R)$ cost and avoiding a full per-round decryption.
- **Vote-privacy overhead.** The pairwise perturbation mechanism adds $\mathcal{O}(NJ)$ client computation and $\mathcal{O}(NKJ)$ KGC computation per round. This is the main asymptotic novelty of ECPFL and is acceptable when N is moderate (cross-silo regime).
- **MASER vs. ECPFL.** Both use multi-key techniques, but MASER relies on partial decryption and growing ciphertexts, while ECPFL/ESAFL use a summed global key S for one-step decryption. ECPFL further reduces per-client upload via consensus voting and amortizes download via prefetching.

Table 2: Per-round computational complexity comparison

Scheme	**Client (per selected client)**	**Server**	**KGC / Authority**
FedAvg+CKKS	$\mathcal{O}(d \log n)$ enc/dec	$\mathcal{O}(Kd)$ HE ops $+\mathcal{O}(d \log n)$ dec	One-time only
MASER	$\mathcal{O}(d \log n + d_w)$ enc/partial-dec/mask	$\mathcal{O}(K^2 dL)$ agg $+\mathcal{O}(KdL)$ merge	$\mathcal{O}(NdL)$ one-time pk aggregation
FedPHE	$\mathcal{O}(sd \log n)$ enc $+\mathcal{O}(d \log n)$ dec	$\mathcal{O}(Ksd \log n)$ HE mult/add	None in reference impl.
ESAFL	$\mathcal{O}(d \log n)$ enc $+\mathcal{O}(d \log n)$ dec	$\mathcal{O}(Kd)$ HE additions	One-time only
ECPFL (proposed)	$\mathcal{O}(d + NJ)$ vote+perturbation $+\mathcal{O}(kn \log n)$ enc amortized $\mathcal{O}(\frac{kn \log n}{R})$ prefetch dec	$\mathcal{O}(KJ)$ vote aggregation $+\mathcal{O}(J \log J)$ consensus $+\mathcal{O}(Kk)$ HE aggregation $+\mathcal{O}(k)$ prefetch delta	$\mathcal{O}(NKJ)$ compensatory perturbation $+\mathcal{O}((N-K)kn \log n)$ compensatory ciphertexts

Table 3: Unified comparison of FL schemes (one effective global model update)

Scheme	**Client dominant cost**	**Server dominant cost**	**KGC online cost**
FedAvg+CKKS	$\mathcal{O}(d \log n)$ enc/dec	$\mathcal{O}(Kd)$ HE ops $+\mathcal{O}(d \log n)$ dec	None
MASER	$\mathcal{O}(d \log n)$ enc/partial-dec	$\mathcal{O}(K^2 dL)$ agg $+\mathcal{O}(KdL)$ merge	One-time pk aggregation
FedPHE	$\mathcal{O}(d \log n)$ enc/dec	$\mathcal{O}(Ksd \log n)$ HE mult/add	None
ESAFL	$\mathcal{O}(d \log n)$ enc/dec	$\mathcal{O}(Kd)$ HE additions	None
ECPFL	$\mathcal{O}(d + NJ)$ vote/perturbation $+\mathcal{O}(kn \log n)$ enc	$\mathcal{O}(KJ + Kk)$ vote + HE agg	$\mathcal{O}(NKJ) + \mathcal{O}((N-K)kn \log n)$

Scheme	**Upload / Download**	**Rounds per update**	**Dropout robust**	**Regime**
FedAvg+CKKS	J / J	1	Limited	Cross-silo
MASER	J / J	3	No	Cross-silo
FedPHE	sJ / J	1	Yes	Cross-device
ESAFL	J / J	1	No	Cross-silo
ECPFL	k / k amortized	1	Partial	Cross-silo